

Git Commands — Complete Developer Guide

A practical reference covering major Git porcelain commands, advanced commands, important flags, and examples.

Source reference: official Git documentation at git-scm.com/docs. This PDF organizes the commands for learning and daily development; it is not a verbatim copy of the Git manual.

1. Core Git Mental Model

Git development normally moves changes through three areas: working tree → staging area/index → commit history.

Typical flow:

```
git status
git diff
git add .
git diff --staged
git commit -m "message"
```

Remember: `git restore` changes files, `git reset` moves/reset Git state, and `git revert` creates a new commit that reverses an earlier commit.

2. Commands and Examples

git

Main Git command; use it to access Git subcommands and global options.

```
git --version
git --help
git -C /path/to/project status
```

git config

Configure Git settings at system, global, or repository level.

```
git config --global user.name "Sasank"
git config --global user.email "you@example.com"
git config --list
git config --global --list
git config user.name
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
git config --global alias.st status
```

git init

Create a new Git repository.

```
git init
git init -b main
git init --bare
```

git clone

Copy an existing repository, including its history.

```
git clone <repository-url>
git clone -b develop <url>
git clone --depth 1 <url>
```

```
git clone --single-branch -b develop <url>
```

git status

Show the state of the working tree and staging area.

```
git status
```

```
git status -s
```

```
git status -sb
```

git add

Stage changes for the next commit.

```
git add app.py
```

```
git add app.py config.py
```

```
git add src/
```

```
git add .
```

```
git add -A
```

```
git add -u
```

```
git add -p
```

git diff

Show differences between working tree, index, and commits.

```
git diff
```

```
git diff --staged
```

```
git diff --cached
```

```
git diff HEAD~2 HEAD
```

```
git diff --stat
```

```
git diff --name-only
```

```
git diff --name-status
```

```
git diff -w
```

```
git diff --word-diff
```

git commit

Create a commit from staged changes.

```
git commit -m "Add login feature"
```

```
git commit -a -m "Fix bug"
```

```
git commit --amend
```

```
git commit --amend -m "Correct commit message"
```

```
git commit --amend --no-edit
```

```
git commit --allow-empty -m "Trigger deployment"
```

git log

View commit history.

```
git log
```

```
git log --oneline
```

```
git log --graph
```

```
git log --decorate
```

```
git log --oneline --graph --decorate --all
```

```
git log -n 5
```

```
git log --author="Sasank"
git log --grep="login"
git log --since="2 weeks ago"
git log --until="2026-08-20"
git log -p
git log --stat
git log --follow -- app.py
```

git show

Inspect a commit or other Git object.

```
git show
git show HEAD
git show <commit>
git show --no-patch HEAD
git show --stat HEAD
git show HEAD:app.py
```

git branch

Create, list, rename, and delete branches.

```
git branch
git branch feature/login
git branch -d feature/login
git branch -D feature/login
git branch -m new-name
git branch -m old-name new-name
git branch -a
git branch -r
git branch -v
git branch -vv
```

git switch

Switch branches; modern alternative to checkout for branch operations.

```
git switch main
git switch -c feature/login
git switch --track origin/develop
```

git checkout

Legacy/general-purpose command for switching branches and restoring files.

```
git checkout main
git checkout -b feature/login
git checkout -- app.py
```

git restore

Restore working-tree files or unstage changes.

```
git restore app.py
git restore --source=HEAD app.py
git restore --staged app.py
```

```
git restore --source=HEAD --staged --worktree app.py
```

git reset

Move HEAD and/or reset the index and working tree. Use carefully.

```
git reset --soft HEAD~1
```

```
git reset HEAD~1
```

```
git reset --hard HEAD~1
```

```
git reset HEAD -- app.py
```

git revert

Create a new commit that reverses an earlier commit.

```
git revert <commit>
```

```
git revert --no-edit <commit>
```

```
git revert <commit1> <commit2>
```

git merge

Combine histories from another branch.

```
git merge feature
```

```
git merge --no-ff feature
```

```
git merge --ff-only feature
```

```
git merge --abort
```

git rebase

Replay commits onto a new base, often used to keep feature history linear.

```
git rebase main
```

```
git rebase -i HEAD~5
```

```
git rebase --continue
```

```
git rebase --abort
```

```
git rebase --skip
```

git cherry-pick

Apply the changes introduced by selected existing commits to the current branch.

```
git cherry-pick <commit>
```

```
git cherry-pick <commit1> <commit2>
```

```
git cherry-pick <first>^..<last>
```

```
git cherry-pick -n <commit>
```

```
git cherry-pick --continue
```

```
git cherry-pick --abort
```

git stash

Temporarily store uncommitted changes.

```
git stash
```

```
git stash push -m "login work"
```

```
git stash -u
```

```
git stash -a
```

```
git stash list
```

```
git stash show -p
```

```
git stash apply
```

```
git stash apply stash@{1}
git stash pop
git stash drop stash@{0}
```

git remote

Manage remote repository connections.

```
git remote
git remote -v
git remote add origin <url>
git remote set-url origin <url>
git remote remove origin
git remote show origin
```

git fetch

Download remote refs and objects without integrating them into the current branch.

```
git fetch
git fetch --all
git fetch --prune
git fetch -p
```

git pull

Fetch and integrate remote changes; integration may be merge or rebase depending on options/config.

```
git pull
git pull --rebase
git pull --ff-only
```

git push

Upload local commits/refs to a remote repository.

```
git push
git push -u origin feature/login
git push origin main
git push origin --delete feature/login
git push --force
git push --force-with-lease
```

git tag

Create and manage tags, commonly for releases.

```
git tag
git tag v1.0.0
git tag -a v1.0.0 -m "Release version 1.0.0"
git show v1.0.0
git tag -d v1.0.0
git push origin v1.0.0
git push origin --tags
```

git rm

Remove tracked files and stage the deletion.

```
git rm app.py
```

```
git rm --cached config.env
```

git mv

Move or rename a tracked file and stage the change.

```
git mv old.py new.py
```

git clean

Remove untracked files/directories. Always preview with -n first.

```
git clean -n
```

```
git clean --dry-run
```

```
git clean -f
```

```
git clean -fd
```

```
git clean -fdx
```

git blame

Show the last commit/author associated with each line.

```
git blame app.py
```

```
git blame -L 20,40 app.py
```

git grep

Search tracked files for text or patterns.

```
git grep "TODO"
```

```
git grep "TODO" HEAD
```

```
git grep -n "TODO"
```

git bisect

Binary-search commit history to find the commit that introduced a regression.

```
git bisect start
```

```
git bisect bad
```

```
git bisect good <commit>
```

```
git bisect bad
```

```
git bisect good
```

```
git bisect reset
```

git reflog

Show local reference movements; useful for recovering from many accidental resets/rebases.

```
git reflog
```

git notes

Attach separate notes to commits without changing the commit itself.

```
git notes add -m "Reviewed by security team"
```

```
git notes show
```

git archive

Create an archive of repository content without the normal working Git metadata.

```
git archive HEAD
```

```
git archive --format=zip HEAD > project.zip
```

git apply

Apply a patch file to the working tree/index.

```
git apply fix.patch
```

```
git diff > changes.patch
```

```
git apply changes.patch
```

git format-patch

Generate patch files from commits.

```
git format-patch HEAD~3
```

git am

Apply patches generated by format-patch.

```
git am 0001-add-login.patch
```

git range-diff

Compare two versions of a commit series, useful after rebasing/reworking patches.

```
git range-diff main...old-branch main...new-branch
```

git shortlog

Summarize commits, commonly grouped by author.

```
git shortlog
```

```
git shortlog -sn
```

git describe

Create a human-readable name for a commit using nearby tags and commit distance.

```
git describe
```

git worktree

Create additional working trees linked to one repository, allowing multiple branches checked out simultaneously.

```
git worktree add ../hotfix hotfix/payment
```

git submodule

Embed another Git repository as a submodule.

```
git submodule add <url> library
```

```
git clone --recurse-submodules <url>
```

```
git submodule update --init --recursive
```

git sparse-checkout

Limit the working tree to selected paths, useful for large repositories.

```
git sparse-checkout init --cone
```

```
git sparse-checkout set src docs
```

git gc

Perform repository housekeeping/optimization.

```
git gc
```

git fsck

Check repository object connectivity and integrity.

```
git fsck
```

git count-objects

Count loose objects and report object database statistics.

```
git count-objects
```

git cat-file

Inspect low-level Git objects.

```
git cat-file -t HEAD
```

```
git cat-file -p HEAD
```

git ls-files

List files tracked by Git and inspect index entries.

```
git ls-files
```

```
git ls-files --stage
```

git ls-tree

List the contents of a tree object.

```
git ls-tree HEAD
```

git rev-parse

Resolve revisions and provide repository/script-friendly information.

```
git rev-parse HEAD
```

```
git rev-parse --abbrev-ref HEAD
```

```
git rev-parse --show-toplevel
```

git rev-list

List commits reachable from a revision.

```
git rev-list HEAD
```

```
git rev-list --count HEAD
```

git merge-base

Find a best common ancestor between commits/branches.

```
git merge-base main feature/login
```

git check-ignore

Explain why a path is ignored.

```
git check-ignore -v app.log
```

git update-index

Advanced manipulation of the Git index.

```
git update-index --assume-unchanged config.json
```

git update-ref

Advanced direct manipulation of Git references.

```
git update-ref refs/heads/test HEAD
```

git symbolic-ref

Read or modify symbolic references.

```
git symbolic-ref --short HEAD
```

git write-tree

Advanced plumbing command: write the current index as a tree object.

```
git write-tree
```

git commit-tree

Advanced plumbing command: create a commit object directly from a tree.

```
git commit-tree <tree>
```

3. Important Flags Quick Reference

Flag	Meaning	Use
-m	message	Provides a commit/tag/stash message depending on the command.
-a	all / tracked changes	Meaning depends on the command; e.g. git commit -a stages tracked modifications/deletions.
-b	branch	Often selects or creates a branch, but meaning is command-specific.
-v / --verbose	verbose	Show more detailed output.
-q / --quiet	quiet	Reduce normal output.
-n / --dry-run or number	command-specific	Meaning varies; e.g. log -n limits commits, clean -n previews removal.
-p / --patch	patch	Show/select changes interactively or display patches, depending on command.
-f / --force	force	Override a safety check; use carefully.
-d / -D	delete	Branch -d safely deletes; -D forces deletion.
-r	remote/recursive	Meaning depends on command.
-u	upstream/update	Meaning depends on command; push -u sets upstream, add -u updates tracked paths.
--amend	amend commit	Modify the previous commit.
--no-edit	keep message	Reuse an existing/generated commit message.
--dry-run	preview	Show what would happen without making changes, where supported.
--all	all	Operate on all matching refs/items, depending on command.
--stat	statistics	Show a summary of changed files and line counts.
--oneline	compact log	Show each commit on one line.
--graph	history graph	Draw an ASCII commit graph.
--decorate	refs	Show branch/tag names attached to commits.
--cached / --staged	index/staging area	Operate on staged/index contents.
--soft	keep index/worktree	For reset: move HEAD while preserving staged and working-tree changes.
--mixed	reset index	For reset: move HEAD and reset the index, leaving working-tree changes; this is reset's default mode.
--hard	reset everything	For reset: reset HEAD, index, and working tree; uncommitted changes can be lost.
--rebase	rebase integration	Use rebase instead of merge where supported.
--abort	cancel operation	Cancel an in-progress merge/rebase/cherry-pick and return to a pre-operation state when possible.
--continue	continue operation	Continue an interrupted operation after resolving conflicts.
--skip	skip item	Skip the current commit/item in operations such as rebase.
--no-ff	no fast-forward	Force a merge commit.
--ff-only	fast-forward only	Refuse the operation unless it can be completed by fast-forwarding.
--force-with-lease	safer force push	Force-update a remote ref only when it still matches your expected remote-tracking state.

4. Most Important Developer Workflows

New project

```
git init -b main
git add .
git commit -m "Initial commit"
```

Clone and create a feature

```
git clone
cd
git switch -c feature/login
```

Review before committing

```
git status
git diff
git add .
git diff --staged
```

Commit and publish

```
git commit -m "Add login feature"
git push -u origin feature/login
```

Update a feature branch with main

```
git fetch origin
git rebase origin/main
git push --force-with-lease
```

Undo safely on a shared branch

```
git revert
```

Recover after an accidental reset

```
git reflog
git reset --hard
```

Inspect branch history

```
git log --oneline --graph --decorate --all
```

5. Safety Notes

- `git reset --hard` can discard uncommitted work.
- `git clean -fdx` can delete ignored files/directories; preview with `git clean -n` first.
- `git push --force` can overwrite remote history; prefer `git push --force-with-lease` when a force push is genuinely required.
- `git branch -D` force-deletes a local branch.
- Before destructive operations, inspect `git status`, `git diff`, and `git reflog` as appropriate.

6. Suggested Learning Order

Begin with: `init`, `clone`, `status`, `add`, `diff`, `commit`, `log`, `show`, `branch`, `switch`, `fetch`, `pull`, `push`, `merge`. Then learn `restore`, `reset`, `revert`, `rebase`, `cherry-pick`, `stash`, `tag`, `reflog`, `blame`, `grep`, `bisect`, `worktree`, `submodule`, and finally Git plumbing commands such as `cat-file`, `ls-tree`, `rev-parse`, `update-ref`, `write-tree`, and `commit-tree`.

7. Command Category Summary

Category	Commands
Setup	<code>git</code> , <code>config</code> , <code>help</code> , <code>bugreport</code>
Create repositories	<code>init</code> , <code>clone</code>
Basic changes	<code>add</code> , <code>status</code> , <code>diff</code> , <code>commit</code> , <code>restore</code> , <code>reset</code> , <code>rm</code> , <code>mv</code>
Branching	<code>branch</code> , <code>switch</code> , <code>checkout</code>
Merging	<code>merge</code> , <code>mergetool</code>
History	<code>log</code> , <code>show</code> , <code>shortlog</code> , <code>describe</code>
Temporary work	<code>stash</code> , <code>worktree</code>
Remote	<code>remote</code> , <code>fetch</code> , <code>pull</code> , <code>push</code>
Tags	<code>tag</code>
Patching	<code>apply</code> , <code>cherry-pick</code> , <code>rebase</code> , <code>revert</code> , <code>format-patch</code> , <code>am</code>
Debugging	<code>bisect</code> , <code>blame</code> , <code>grep</code>

Category	Commands
Repository inspection	diff, show, range-diff, difftool
Maintenance	clean, gc, fsck, reflog
Advanced/plumbing	cat-file, ls-files, ls-tree, rev-parse, rev-list, merge-base, update-ref, update-index, symbolic-ref, write-tree, commit